

C++ 연습 문제: 15. 멀티스레딩과 동시성

1. C++11에서 표준 라이브러리를 통해 새로운 독립적인 스레드를 생성하기 위해 사용하는 클래스는 무엇인가? (하)

1. `std::process`
2. `std::thread`
3. `std::task`
4. `std::async_future`

2. 생성된 `std::thread` 객체가 작업을 완전히 마칠 때까지 호출한 스레드가 대기하도록 만들기 위해 호출하는 멤버 함수는 무엇인가? (하)

1. `wait()`
2. `join()`
3. `detach()`
4. `stop()`

3. `std::thread` 객체가 생성된 후 `join()`이나 `detach()`를 호출하지 않은 상태로 소멸될 때 발생하는 현상으로 올바른 것은? (중)

1. 스레드가 자동으로 백그라운드 전환된다.
2. 컴파일 오류가 발생한다.
3. 프로그램이 즉시 강제 종료(`std::terminate`)된다.
4. 메모리 누수가 발생하지만 실행은 유지된다.

4. 둘 이상의 스레드가 동시에 같은 메모리 위치에 접근하여 최소한 하나가 쓰기 작업을 수행할 때 발생하는 미정의 동작(Undefined Behavior)을 무엇이라고 하는가? (하)

1. 교착 상태 (Deadlock)
2. 데이터 레이스 (Data Race)
3. 라이브 락 (Livelock)
4. 기아 상태 (Starvation)

5. 임계 구역(Critical Section)에 여러 스레드가 동시에 진입하는 것을 막기 위해 상호 배제(Mutual Exclusion)를 제공하는 가장 기본적인 표준 동기화 도구는 무엇인가? (하)

1. `std::mutex`
2. `std::atomic`
3. `std::future`
4. `std::condition_variable`

6. `std::mutex`를 수동으로 `lock()`하고 `unlock()`하는 번거로움을 없애고, RAII 원칙을 적용하여 범위를 벗어날 때 자동으로 락이 해제되도록 돕는 대표적인 클래스는 무엇인가? (중)

1. `std::unique_lock`

2. `std::lock_guard`
3. `std::shared_lock`
4. `std::scoped_lock`

7. 두 개 이상의 뮤텝스를 동시에 잠그려 할 때 발생하는 교착 상태(Deadlock)를 안전하게 예방하기 위해 여러 락을 한 번에 원자적으로 잠그는 표준 함수는 무엇인가? (상)

1. `std::lock`
2. `std::mutex_all`
3. `std::try_lock`
4. `std::sync_locks`

8. 락을 소유한 상태에서 특정 조건이 충족될 때까지 스레드를 대기시키고 다른 스레드의 신호(Notify)를 받아 깨어날 수 있게 해주는 동기화 도구는 무엇인가? (중)

1. `std::condition_variable`
2. `std::promise`
3. `std::async`
4. `std::atomic`

9. 락(Mutex)을 사용하지 않고 하드웨어 레벨에서 원자적(Atomic) 연산을 수행하여 경합 비용을 줄여주는 표준 템플릿 클래스는 무엇인가? (중)

1. `std::secure`
2. `std::atomic`
3. `std::volatile`
4. `std::mutex_free`

10. `std::atomic` 변수에서 동시성 안전성을 보장하며 현재 값을 지정된 값으로 바꾸고 이전 값을 안전하게 반환하는 대표적인 연산 함수는 무엇인가? (상)

1. `fetch_add`
2. `compare_exchange`
3. `exchange`
4. `swap_atomic`

11. 비동기 작업의 결과를 나중에 받아올 수 있도록 결과 값을 담아두는 공용체 역할을 하는 미래 객체는 무엇인가? (하)

1. `std::promise`
2. `std::future`
3. `std::packaged_task`
4. `std::async`

12. 함수나 작업을 비동기로 실행하고 그 결과를 `std::future`를 통해 쉽게 받을 수 있도록 돕는 대표적인 표준 고수준 비동기 함수는 무엇인가? (중)

1. `std::thread`
2. `std::async`

3. `std::fork`
4. `std::invoke_async`

13. `std::async`의 실행 정책(Launch Policy) 중, 작업이 반드시 별도의 새로운 스레드에서 즉시 비동기로 실행되도록 강제하는 값은 무엇인가? (상)

1. `std::launch::deferred`
2. `std::launch::async`
3. `std::launch::immediate`
4. `std::launch::threaded`

14. 전역 변수나 정적 변수와 유사하지만, 프로그램 내 각 스레드마다 완전히 독립적인 메모리 공간을 가지도록 지정하는 키워드는 무엇인가? (중)

1. `static`
2. `extern`
3. `thread_local`
4. `volatile`

15. 뮤텍스의 소유권을 유연하게 해제했다가 다시 잠그거나, `std::condition_variable`과 함께 사용하기에 적합한 다목적 고급 락 관리 클래스는 무엇인가? (상)

1. `std::lock_guard`
2. `std::unique_lock`
3. `std::atomic_lock`
4. `std::Simple_lock`

16. 교착 상태(Deadlock)가 발생하기 위한 4가지 필수 조건에 해당하지 않는 것은 무엇인가? (상)

1. 상호 배제 (Mutual exclusion)
2. 점유와 대기 (Hold and wait)
3. 비선점 (No preemption)
4. 무작위 대칭 분산 (Random symmetric distribution)

17. 멀티스레드 환경에서 지역 정적 변수를 초기화할 때, C++11 이후 컴파일러가 스레드 안전성(Thread-safety)을 완벽하게 보장해 주는 특성을 무엇이라고 하는가? (중)

1. 스레드 안전 지역 정적 초기화 (Thread-safe local static initialization)
2. 원자적 변수 초기화 (Atomic variable initialization)
3. 정적 메모리 바인딩 (Static memory binding)
4. 락 프리 초기화 (Lock-free initialization)

18. 여러 스레드가 동시에 읽기 작업은 마음대로 수행하되, 쓰기 작업 시에는 단 하나의 스레드만 독점적으로 진입하게 해주는 읽기-쓰기 락(Read-Write Lock)을 제공하는 클래스는 무엇인가? (중)

1. `std::shared_mutex`

2. `std::recursive_mutex`
3. `std::timed_mutex`
4. `std::atomic_mutex`

19. 스레드가 생성된 이후 주 스레드와의 연동 관계를 끊고, 백그라운드에서 독립적으로 실행되도록 만드는 `std::thread`의 멤버 함수는 무엇인가? (하)

1. `join()`
2. `detach()`
3. `release()`
4. `disconnect()`

20. 멀티스레딩 프로그램 설계 시, 서로 다른 스레드가 논리적으로 완전히 다른 독립된 변수를 수정하고 있음에도 불구하고 동일한 캐시 라인(**Cache line**)에 위치해 있다는 이유로 성능이 극도로 저하되는 현상을 무엇이라고 하는가? (상)

1. 캐시 미스 (Cache Miss)
2. 메모리 누수 (Memory Leak)
3. 거짓 공유 (False Sharing)
4. 페이지 fault (Page Fault)